api0

GUIDE 04

Architecture et modèles d'authentification supportés par api0

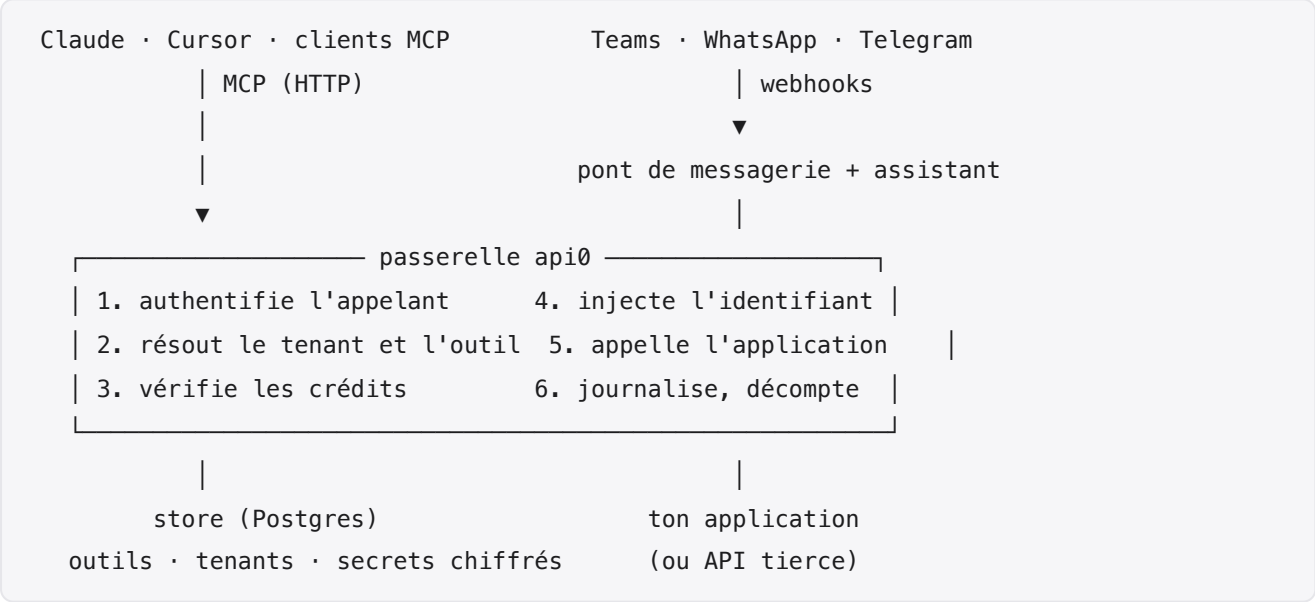
Le guide technique : comment une requête traverse api0, de Claude, Teams ou WhatsApp jusqu'à ton application, et chaque modèle d'authentification en entrée et en sortie, avec quand choisir lequel.

Mis à jour le 7 novembre 2026 · Lire en ligne : api0.ai/fr/guides/architecture-et-auth

---

Ce guide s'adresse aux équipes techniques et sécurité qui évaluent api0. Il décrit le chemin d'une requête, puis les deux questions d'authentification que toute passerelle d'IA doit résoudre : **qui appelle**, et **avec quel identifiant on appelle ton application**.

Vue d'ensemble



Trois composants, écrits en **Rust** :

- **La passerelle** reçoit chaque appel, l'authentifie, le route et le journalise. C'est le seul point de passage vers tes systèmes.
- **Le store** garde le registre des outils, les tenants, les identifiants (chiffrés) et le registre d'usage.
- **Le pont de messagerie** reçoit les webhooks de Teams, WhatsApp et Telegram et fait tourner l'assistant, qui appelle les mêmes outils via la passerelle.

La passerelle MCP

`/mcp` est un serveur **Model Context Protocol** standard sur HTTP (Streamable HTTP). Rien de propriétaire côté protocole : tout client MCP fonctionne.

Méthode	Comportement
`initialize`	Négocie la version du protocole, annonce les outils, renvoie les instructions du tenant
`tools/list`	Les outils du tenant de l'appelant, et seulement eux
`tools/call`	Route vers l'application, décompte, renvoie le résultat
`ping`	Réponse vide

Versions du protocole acceptées : `2025-06-18`, `2025-03-26`, `2024-11-05`.

Instructions du tenant. `initialize` renvoie un texte (4 000 caractères max) que le client place devant le modèle : des règles d'usage communes (charger un outil avant de l'appeler, prendre les identifiants dans un outil de listing plutôt que de les deviner…), le texte propre au tenant (2 000 caractères max) et, pour chaque groupe d'outils, les outils de départ.

Erreurs. Un refus de ton application revient comme un **résultat** marqué `isError`, avec le message de ton application : le modèle le lit et peut corriger. Les erreurs JSON-RPC sont réservées aux refus d'api0 (outil inconnu, paramètres invalides, crédits épuisés). **Un appel en échec n'est jamais facturé.**

Ce qu'est un outil

Un outil est une entrée du registre, rattachée à un tenant :

Champ	Rôle
nom, description, schéma d'entrée	Ce que le modèle voit
URL de l'application	Cible, avec des `{paramètres}` substitués
verbe HTTP	Défini → appel REST. Vide → l'application parle MCP nativement
content-type, modèle de corps	Pour les APIs dont le corps n'est pas l'objet d'arguments (JSON Patch…)
en-têtes fixes	Constantes propres à l'outil
transmission d'identité	Envoyer ou non l'email et le tenant de l'utilisateur
coût, timeout	Décompte et patience

Importer une spec OpenAPI crée ces entrées automatiquement. Les **modèles de corps** couvrent les cas difficiles sans code : un champ optionnel non fourni disparaît du corps (une opération JSON Patch entière dans un tableau, une seule clé dans un objet). Les **noms de paramètres** que les clients MCP refusent (`$filter`, `page[size]`) sont exposés sous un nom sûr et renvoyés à l'application sous leur vrai nom.

Ordre des en-têtes

Appliqués dans cet ordre, le dernier gagne :

1. **identité api0** (`X-User-Email`, `X-Tenant-Id`) — seulement si la transmission d'identité est activée ;
2. **en-têtes fixes** de l'outil ;
3. **authentification sortante** — gagne toujours en cas de collision ;
4. `Content-Type`.

La passerelle n'envoie jamais de secret interne à une application.

Authentification entrante : qui appelle ?

Modèle	Utilisé par	Détail
OAuth 2.1 + PKCE	Claude et les clients MCP	Le client s'enregistre avec le Client ID du tenant ; l'utilisateur se connecte ; la passerelle émet un jeton lié à cet utilisateur et à ce tenant
Connexion Google	Par défaut	Pour le tableau de bord et la connexion des utilisateurs
Ton fournisseur d'identité (OIDC)	Entreprises	Entra ID ou tout IdP OIDC configuré par tenant : tes employés se connectent avec leur compte, MFA et accès conditionnel compris. Le secret client de l'IdP est chiffré
Clé API	Intégrations serveur à serveur	Une clé par tenant ou par consommateur
Liaison de compte messagerie	Teams, WhatsApp, Telegram	Au premier message, l'utilisateur reçoit un lien de courte durée pour relier son identifiant de messagerie à son compte

Agir pour un utilisateur final

Une application peut appeler la passerelle avec sa clé **pour le compte d'un de ses utilisateurs**, en le nommant. La passerelle vérifie avant tout appel que c'est permis : le propriétaire de la clé, toujours ; une clé de consommateur, jamais pour quelqu'un d'autre ; une clé du tenant, seulement pour une personne qui a elle-même rejoint ce tenant (par le consentement OAuth ou sa propre clé). Être ajouté comme membre ne suffit pas. Sinon : `403`.

Authentification sortante : avec quel identifiant appeler ton application ?

Chaque tenant a une **connexion par défaut**, et peut en ajouter d'autres, nommées, quand certains outils visent un service qui demande une autre authentification.

Mode	Ce que reçoit l'application	Quand le choisir
`none`	Rien	API publique

Mode	Ce que reçoit l'application	Quand le choisir
Jeton d'identité api0 (`google_sa` , sans clé)	Un jeton d'identité Google signé par le compte de service d'api0	**Tes propres applications.** Aucun secret à stocker
`google_sa` avec ta clé	Un jeton d'identité signé par **ton** compte de service	Applications Google Cloud (Cloud Run…) avec ton propre IAM
`static_bearer`	`Authorization: Bearer <jeton>`	API tierce à clé unique
`header_injection`	Les en-têtes que tu définis	APIs à en-têtes propriétaires
`client_credentials`	Un jeton obtenu par OAuth client credentials	APIs qui délivrent des jetons de service
`per_user` — secret personnel	L'identifiant de **chaque** utilisateur : `Basic` (PAT Azure DevOps), `Bearer` , ou en-tête brut	Attribution et droits par personne
`per_user` — OAuth	Le jeton OAuth de **chaque** utilisateur, rafraîchi par api0	Systèmes à connexion OAuth (ex. Microsoft)

Le jeton d'identité api0 en détail

Pour une application que tu opères, le plus simple est souvent de ne stocker **aucun** secret : la passerelle signe chaque appel avec le compte de service d'api0.

Un seul signataire pour tous les tenants : c'est donc l'**audience** qui les sépare, et le tenant ne la choisit pas. La passerelle la construit à partir de l'origine de l'URL de l'application et de l'identifiant du tenant appelant :

```
https://api.exemple.com/orders  →  aud = https://api.exemple.com/api0/tenant/<id du tenant>
```

Seule l'origine est gardée : aucun chemin, aucune requête ne peut fabriquer l'audience d'un autre tenant. Ton application vérifie deux valeurs, une fois pour toutes :

Vérification	Valeur
`aud`	`https://<ton origine>/api0/tenant/<ton tenant>`
`email` (signataire)	le compte de service d'api0

Épingler le signataire est obligatoire : n'importe quel compte de service Google peut émettre un jeton pour n'importe quelle audience.

Identifiants par personne

C'est le modèle à préférer dès que l'attribution compte.

- **Secret personnel.** L'utilisateur ajoute une fois son jeton (par exemple un PAT Azure DevOps). Il est chiffré et **ne peut plus être relu**, seulement remplacé. Une URL de vérification optionnelle permet de contrôler à qui appartient le jeton.

- **OAuth.** L'utilisateur connecte une fois son compte. api0 garde le jeton de rafraîchissement chiffré et renouvelle l'accès. Le paramètre `state` du flux OAuth est à usage unique et court (protection CSRF).
- **Sans identifiant** : l'utilisateur reçoit un résultat d'outil qui lui explique comment l'ajouter. **Jamais** d'appel fait au nom de quelqu'un d'autre.

Résultat : le système cible voit la vraie personne, applique ses droits et lui attribue l'action.

Protection des secrets

- Chiffrement **AES-256-GCM**, scellé selon le tenant et l'usage du secret : un secret copié vers un autre contexte ne s'ouvre pas.
- Les secrets ne sont jamais renvoyés au client MCP, ni au modèle, ni au tableau de bord (lecture des métadonnées seulement).
- Sans clé de chiffrement configurée, le store **refuse** d'enregistrer un identifiant plutôt que de le garder en clair.

Données personnelles

La transmission de l'identité (`X-User-Email` , `X-Tenant-Id`) est un réglage **par outil**. Elle dit pour qui est l'appel, pas qu'il vient d'api0 : une application qui doit le savoir vérifie l'identifiant sortant (le jeton signé). **Désactive-la pour tout outil pointé vers un tiers** : les emails de tes utilisateurs restent dans la plateforme.

Usage, crédits et journal

- Chaque appel est journalisé : outil, utilisateur, tenant, statut, latence.
- Chaque appel réussi décompte des crédits ; un échec n'est jamais facturé.
- Mode de facturation par tenant : **le tenant paie**, ou **chaque personne paie** avec son plan.

Choisir son modèle : l'arbre de décision

1. **L'application est à toi ?** → jeton d'identité api0. Rien à stocker.
2. **Les actions doivent être attribuées à la personne, ou ses droits appliqués ?** → `per_user` (OAuth si le système le permet, sinon secret personnel).
3. **API tierce avec une clé de service ?** → `static_bearer` , `header_injection` ou `client_credentials` , et transmission d'identité désactivée.
4. **Des outils visent des services différents ?** → une connexion nommée par service.